



Experiment-8

Student Name: Pranav Chauhan

Branch: BE CSE (cyber security)

Semester: 5th

Subject Name: Computer Networks

UID: 24BCY70004

Section/Group: 24AIT_KRG-G2

Date: 16/09/2026

Subject Code: 24CSH-337

1. Aim/Overview of the practical:

To implement **TCP socket communication using Python** by creating a client-server application that establishes a TCP connection and allows the client and server to exchange messages.

2. Software/Tools Required:

- Python 3.x
- PyCharm / VS Code
- Python socket module
- Windows/Linux operating system

3. Theory:

A) TCP Socket Communication

TCP (Transmission Control Protocol) is a connection-oriented and reliable communication protocol used for exchanging data between two devices. In socket programming, a TCP connection is established between a client and a server before data is exchanged. TCP ensures that data is delivered reliably and in the correct order.

B) Python Socket Programming

Python provides the built-in socket module for network communication. A socket acts as an endpoint through which two programs can communicate. The program creates a socket using `socket.AF_INET` for IPv4 communication and `socket.SOCK_STREAM` for TCP communication.

C) Client-Server Model

In the client-server model, the server waits for incoming connection requests, while the client initiates the connection. The server binds



itself to an IP address and port number, listens for connections, and accepts the client's request. In this experiment, the server uses 127.0.0.1 and port 5000, where 127.0.0.1 represents the local computer.

D) Data Transmission

After establishing the connection, the client can send messages to the server using the `send()` function. The message is converted from a string into bytes using `encode()`. The server receives the data using `recv()` and converts it back into a string using `decode()`. Similarly, the server can send a reply back to the client.

E) Connection Termination

The communication continues until the client sends the message "Over". After the communication is completed, both the client and server close their sockets using `close()`, terminating the TCP connection properly.

4. Steps for Experiment/Practical:

- 1) Install Python and open **PyCharm/VS Code**.
- 2) Create a new project and create two Python files: `server.py` and `client.py`.
- 3) Import the `socket` module in both files.
- 4) In `server.py`, create a TCP socket using `socket.AF_INET` and `socket.SOCK_STREAM`.
- 5) Bind the server socket to IP address 127.0.0.1 and port 5000.
- 6) Use `listen()` to make the server wait for incoming connection requests.
- 7) Use `accept()` to accept the connection from the client.
- 8) In `client.py`, create a TCP socket using `socket.AF_INET` and `socket.SOCK_STREAM`.
- 9) Connect the client to 127.0.0.1 on port 5000.
- 10) Run `server.py` first and wait for the server to start.
- 11) Run `client.py` and establish the connection with the server.
- 12) Enter a message on the client side and send it to the server.
- 13) The server receives the message using `recv()` and displays it.
- 14) Enter a reply on the server side and send it back to the client.
- 15) The client receives and displays the server's reply.
- 16) Continue the communication until the message "**Over**" is entered.
- 17) Close both client and server sockets after communication is completed.



5. Output:

```
server.py X
server.py > ...
22  # If no client has connected, the program will pause over here only.
23  ''' This command waits for a client.
24
25  The server stops here until a client connects.'''
26
27
28  print("Client connected:", client_address)
29
30  while True:
31      message = client_socket.recv(1024).decode()
32      ## Will decode the coming message from the client of 1024 bytes.
33      # It attempts to receive up to 1024 bytes from the TCP connection.
34
35      print("Client:", message)
36
37      if message == "Over":
38          break
39
40      ## Server Reply
41      reply = input("Enter reply: ")
42
43      client_socket.send(reply.encode())
44
45  client_socket.close()
46  server_socket.close()
47
48  print("Connection closed")
```

```
client.py X
client.py > ...
1
2  import socket
3
4  client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
5
6  ## SOCK_STREAM: represents TCP Segment / TCP Data
7
8  client_socket.connect(("127.0.0.1", 5000))
9
10 ## Client connecting to server with 127.0.0.1 on port 5000
11
12 print("Connected to server")
13
14 while True:
15     message = input("Enter message: ")
16
17     client_socket.send(message.encode()) ##Encode(): Converts raw string to byte
18
19     if message == "Over":
20         break
21
22     reply = client_socket.recv(1024).decode()
23
24     print("Server:", reply)
25
26 client_socket.close()
27
28 print("Connection closed")
```

```
Problems Output Debug Console Terminal Ports
PS C:\PranavData\main data\CU\notes\5th sem\cn\Practicals\exp-8\TCP_Socket_Experiment> & C:\Users\Pranav\AppData\Local\Programs\Python\Python314\python.exe "c:/PranavData/main data/CU\notes/5th sem/cn/Practicals/exp-8/TCP_Socket_Experiment/server.py"
Server started
Waiting for a client...
Client connected: ('127.0.0.1', 64456)
Client: hello my name is pranav
Enter reply: hi
[]

PS C:\PranavData\main data\CU\notes\5th sem\cn\Practicals\exp-8\TCP_Socket_Experiment> & C:\Users\Pranav\AppData\Local\Programs\Python\Python314\python.exe "c:/PranavData/main data/CU\notes/5th sem/cn/Practicals/exp-8/TCP_Socket_Experiment/client.py"
Connected to server
Enter message: hello my name is pranav
Server: hi
Enter message: []
```



6. Result

The TCP client-server application was successfully implemented using Python sockets. The client established a connection with the server, exchanged messages, and received replies successfully. The connection was terminated after sending "**Over**".

7. Learning Outcomes

- Understand the basics of TCP socket communication.
- Learn to create TCP sockets using Python.
- Understand the client-server communication model.
- Learn to send and receive messages using sockets.
- Understand how to establish and terminate a TCP connection.